

kube-bench Results

Detailed kube-bench CIS Kubernetes Benchmark-oriented review, interpretation framework, limitations, remediation themes, and retest plan.

Field	Value
Document Type	Benchmark Results Summary
Author	Jagriti Banerjee
Project Scope	Private Kubernetes DevSecOps lab using Docker, Kind, Calico, Trivy, Falco, OPA/Gatekeeper, ArgoCD, and kube-bench
Audience	Security engineers, DevSecOps reviewers, recruiters, and technical interviewers
Classification	Private lab / portfolio evidence

Reviewer snapshot

Review Dimension	Included in Document
Technical depth	Control rationale, validation commands, expected results, risk interpretation, and remediation guidance.
Evidence model	Screenshot references, manifest fields, scanner outputs, and repeatable kubectl commands.
Professional use	Can be used as a GitHub portfolio artifact, interview talking point, and internal-style security record.
CIS alignment	Includes a dedicated CIS Kubernetes Benchmark section with lab-context interpretation.

Scope statement

This document is based on controlled, authorized testing performed inside a private lab environment. It does not represent a production certification and does not include public or third-party targets. The focus is professional evidence, repeatable validation, and security engineering reasoning.

Spacing and readability note

The document is intentionally laid out with compact paragraphs, split tables, and dedicated evidence pages so that detail is preserved without crowded pages or large blank gaps.

Document coverage

Coverage Area	Included
Assessment narrative	Executive summary, scope, methodology, technical analysis, and validation interpretation.
Evidence references	Relevant screenshots, command outputs, reports, and expected retest outcomes.
Remediation planning	Priority-based actions, production notes, and follow-up recommendations.
Benchmark context	CIS Kubernetes Benchmark alignment and lab-specific interpretation.

1. Executive Summary

This document summarizes kube-bench usage and CIS Kubernetes Benchmark-oriented review for the Enterprise DevSecOps lab. The goal is to demonstrate awareness of benchmark-driven Kubernetes hardening while clearly explaining the limitations of applying production benchmarks to a Kind-based private lab.

2. CIS Kubernetes Benchmark Section

CIS describes the Kubernetes Benchmark as consensus-based secure configuration guidance for Kubernetes. In this lab, CIS Benchmark alignment is used as an assessment reference and learning framework. Because the cluster is a Kind-based private lab rather than a production kubeadm or managed service environment, benchmark results are interpreted with environment context. The expected professional practice is to document PASS, FAIL, WARN, and not-applicable outcomes; assign owners to applicable failures; retest after remediation; and keep benchmark artifacts in the evidence folder.

3. Execution Context

Field	Value
Tool	kube-bench
Benchmark basis	CIS Kubernetes Benchmark secure configuration guidance.
Cluster type	Kind cluster running inside private Ubuntu VM.
Purpose	Portfolio-grade benchmark evidence and hardening interpretation.
Important limitation	Kind does not expose every production control plane and node configuration in the same way as kubeadm or managed clusters.
Evidence artifact	028-kube-bench-cis-results.png and security/reports/kube-bench-results.txt.

4. Methodology

4.1 How kube-bench was run

The kube-bench job manifest was applied to the cluster. The resulting pod logs were collected and stored as evidence. Findings were interpreted in relation to the Kind environment and mapped to broader Kubernetes hardening themes.

```
kubectl apply -f kube-bench/job.yaml
kubectl get pods -l app=kube-bench
POD=$(kubectl get pods -l app=kube-bench -o jsonpath='{.items[0].metadata.name}')
kubectl logs "$POD" > security/reports/kube-bench-results.txt
kubectl logs "$POD" | less
```

4.2 Evidence handling

The raw kube-bench log should be stored as a report artifact and reviewed line by line. For portfolio documentation, the important point is not only the result count, but the ability to explain which checks are applicable, which are Kind-specific limitations, and which controls require production validation.

5. Result Interpretation

Result	Meaning	Required Action
PASS	Observed configuration matches the check expectation.	Keep evidence and retest after cluster changes.
FAIL	Observed configuration does not match the check expectation.	Review applicability, remediate if applicable, or document exception.
WARN	Manual review or environment-specific interpretation required.	Assign reviewer and document decision.
INFO	Contextual result, not necessarily a defect.	Use for understanding posture.
Not applicable	Check does not apply to Kind or the deployment type.	Document why it is not applicable.

6. Benchmark Control Families

Control Family	Review Focus	Lab Interpretation
Control plane	API server, scheduler, controller manager flags and permissions.	Kind may not match production deployment layout; use as educational reference.
Worker node	kubelet configuration, file permissions, authentication and authorization.	Useful for node-hardening themes and future kubeadm lab work.
etcd	Data store configuration, TLS, and encryption at rest.	Managed/production clusters need separate validation.
Policies	Admission controls, pod security, audit policy, RBAC.	Mapped to Pod Security, RBAC, Gatekeeper, and manifest scans.
Network	Segmentation and secure communication themes.	Mapped to Calico NetworkPolicy evidence.

7. Remediation Themes

- Convert applicable FAIL/WARN results into tracked remediation items with owner, priority, and retest command.
- Use benchmark findings to strengthen admission policies and CI security gates.
- Review API server, kubelet, audit logging, and secrets encryption differently for production or managed clusters.
- Keep kube-bench reports in security/reports to prove benchmark review occurred.
- Run kube-bench after cluster recreation, Kubernetes version upgrade, or major control plane changes.

8. Evidence Highlight

9. Retest and Closure Plan

Step	Activity	Evidence
1	Run kube-bench job.	Pod logs generated and saved.
2	Classify results.	PASS, FAIL, WARN, INFO, and not-applicable register.
3	Prioritize applicable failures.	Owner, severity, and remediation task assigned.
4	Apply remediation or exception.	Manifest, cluster setting, or documented exception updated.
5	Retest.	Updated kube-bench report showing improvement or justified exception.
6	Archive evidence.	Report stored in security/reports and linked in evidence index.

10. Production Readiness Notes

- For managed Kubernetes, review cloud-provider benchmark guidance in addition to the generic Kubernetes Benchmark.
- For kubeadm clusters, review actual component configuration files and systemd unit settings.
- Enable audit logging and secrets encryption at rest where supported.
- Use benchmark checks as one layer of assurance alongside runtime detection, SAST/SCA, image scanning, admission policies, and manual architecture review.

Quality Assurance and Evidence Preservation

QA Item	Required Practice
Evidence naming	Use stable screenshot names and link each item to a control or retest step.
Command repeatability	Store commands in documentation so another reviewer can reproduce validation.
Exception handling	Any accepted risk should include owner, reason, expiry date, and compensating control.
Retest discipline	Do not close a remediation item until the expected negative and positive tests both pass.
CI/CD linkage	Where possible, convert manual validation into security gates or scheduled checks.

Reviewer notes for this Kubernetes security area

This document intentionally combines implementation evidence with risk interpretation. The objective is to show not only that commands were executed, but also why each control matters, how it reduces attack surface, and how the result should be retested after future changes.

References and Standards Basis

Reference	Use in this Document
CIS Kubernetes Benchmark	CIS describes its Kubernetes Benchmark as a product of a community consensus process with secure configuration guidelines for Kubernetes.
Kubernetes RBAC documentation	Kubernetes RBAC uses Role, ClusterRole, RoleBinding, and ClusterRoleBinding objects to manage authorization.
Kubernetes NetworkPolicy documentation	NetworkPolicy governs how pods communicate with each other and with network endpoints.
Kubernetes Pod Security Standards	The restricted profile is intended for strongly hardened workload execution.

Reference URLs used during document preparation: <https://www.cisecurity.org/benchmark/kubernetes>, <https://kubernetes.io/docs/reference/access-authn-authz/rbac/>, <https://kubernetes.io/docs/concepts/services-networking/network-policies/>, and <https://kubernetes.io/docs/concepts/security/pod-security-standards/>.